You are trying to convert a string representation to integer. In your use-case scenario, you know that the string will never contain any special characters. However, the library function you are using cannot make this assumption, since it needs to be generic to handle any string, including those that contain special characters. So it performs a redundant check for special characters, which is not needed in your use-case scenario. This is a simple example of the library function being bloated.

While this is an example of bloat in operations that are executed, let us consider another example which results in extra memory being used. Your code needs a date object, but you know that you don't need the day of the week corresponding to the date. However your 'Date' class from the library allocates an additional field for storing the 'day of the week' corresponding to the date. This results in additional memory consumption, which is not needed by your use-case.

Both these forms of inefficiency are examples of 'software bloat', which impacts performance adversely. It is a well-known fact that, unfortunately, many of today's large commercial software applications suffer from software bloat to a considerable extent. This leads to poor scalability and excessive use of hardware resources to meet the performance needs of the bloated software, compared to a lean and efficient version of the same software. There is a trade-off programmers make in using libraries and frameworks extensively, as opposed to writing custom code. Frameworks and libraries contribute to software bloat (unless they have been written very carefully, providing multiple specialised APIs—which, unfortunately, is not the case today). But they aid rapid application development. Software developers need to understand the performance penalties associated with the various libraries and frameworks they leverage in their application.

Types of software bloat

The term 'bloat' refers to a condition wherein a system is not functioning in an efficient manner. The inefficiency may be due to redundant operations being performed (execution bloat), or redundant memory being allocated (memory bloat). The latter also includes memory that is allocated and not used, or memory not reclaimed even after it is no longer live, etc. Consider a Java application, which has automatic memory management. A memory leak in a Java application is a typical example of memory bloat, since an object is not getting freed up though it is no longer needed. This situation, if excessively found, can cause the garbage collector to run out of heap space (since it cannot collect the leaked objects, it cannot free up memory and make it available for the application) and thus causes the application to crash.

Execution bloat is typically associated with redundant operations being performed while there is a more efficient way to achieve the same end result. Consider a function that uses 'bubble sort' to generate sorted output. This obviously suffers from execution inefficiency, since 'quick-sort' would have been more efficient. This is an example of execution bloat. The earlier example we looked at was of a framework's string-conversion function that checks for special characters.

By now, you may wonder, "Isn't software bloat supposed to be removed by compilers/runtime and garbage collectors?" Well, the answer is "Yes, to some extent." However, since today's software applications are very complex, it becomes increasingly difficult for compilers, JIT and runtime to detect and eliminate bloat, since bloat is not caused by operations in one single function, method or class. Bloat occurs as the application state propagates across from custom code written specifically for your application over many layers of libraries or /frameworks, and it becomes difficult to detect and eliminate cross-layer bloat automatically.

Currently, there is increasing attention being given to understanding the causes of software bloat. A summary of the nature and causes of bloat can be found in an article in IEEE software titled, 'Four Trends Leading to Java Runtime Bloat' by Nick Mitchell and others. While this article focuses on Java, many of the causes of bloat that it describes are equally applicable to all modern 'framework-intensive' languages like JavaScript, .NET, Python, etc. Well, the question: "Is bloat inevitable because of the object-oriented paradigms and focus on re-use in modern programming languages?" will be discussed in the next column.

My 'must-read book' for this month

This month's 'must-read book' suggestion comes from our reader, Keshavan. He suggests the book, 'The Design of Design—Essays from a computer scientist' by Frederick P Brooks. According to Keshavan, "This is a great book if you ever wanted to know what makes an efficient design, and is a worthy successor to the software developer classic, 'The Mythical Man-Month' by the same author. The book discusses the traditional waterfall model of design, goes on to examine its flaws, and the alternatives to it. It then discusses collaborative design, which is increasingly important in today's 'flat' world, and concludes with a number of case studies in design." Thank you, Keshavan, for the recommendation.

If you have a favourite programming book or article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so I can mention it in this column. This would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Till we meet again next month, happy programming and here's wishing you the very best! END

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a researcher in IBM India Research Labs. Her interests include compilers, multi-core technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at *http://www.linkedin.com/groups?home=&gid=2339182*.